

Máquina VOLE

y fundamentos de sistemas operativos

Juancarlos Monsalve



Apuntes
Compiladores e
intérpretes

PC

IR

13 de Agosto

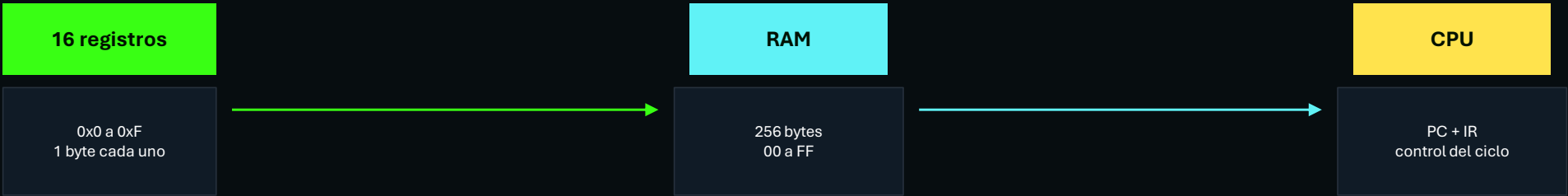
Mapa general de la presentación

Qué hay que dominar para resolver ejercicios VOLE y entender el S.O.

01	Máquina VOLE	Registros, RAM, PC e IR
02	Ciclo FETCH	Qué pasa en cada paso de ejecución
03	Formato de instrucción	OP, R, X, Y y cómo decodificar
04	Ejemplos VOLE	LOAD, LOADX, STORE, JMP, OR, HALT
05	Sistemas operativos	JCL, E/S, servicios y máquinas virtuales

1. Máquina VOLE: arquitectura básica

Una máquina didáctica para ver cómo se ejecuta el lenguaje máquina



Registros: guardan valores temporales para operar y mover datos.

RAM: contiene programa y datos. Cada celda tiene una dirección hexadecimal.

PC indica dónde leer; IR guarda la instrucción de 2 bytes que se ejecutará.

20	21	22	23	24	25
20	C0	30	04	00	00



Idea clave: cada instrucción ocupa 2 bytes; por eso normalmente el PC avanza de 2 en 2.

2. Formato de instrucción

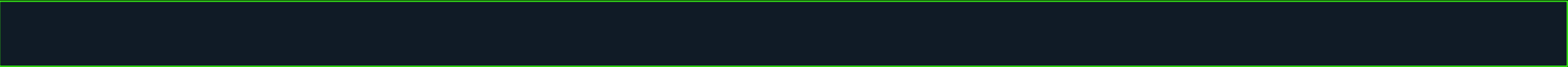
Toda instrucción VOLE se lee como 4 dígitos hexadecimales



Ejemplo: 20C0

2 = LOADX
R = 0
XY = C0
Resultado: r0 ← C0

OP	Nombre	¿Qué hace?
1	LOAD	$rR \leftarrow \text{mem}[XY]$
2	LOADX	$rR \leftarrow XY$
3	STORE	$\text{mem}[XY] \leftarrow rR$
7	OR	$rR \leftarrow rS \text{ OR } rT$
B	JMP	si $rR = r0$, $PC \leftarrow XY$
C	HALT	Detiene ejecución



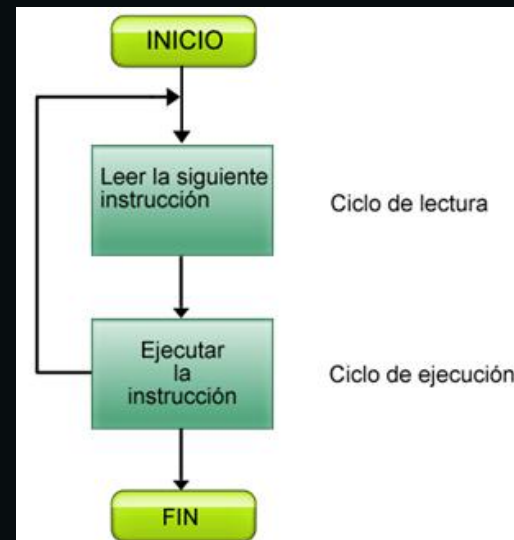
3. Ciclo FETCH

La máquina repite el mismo ciclo hasta llegar a HALT



Qué se anota en cada ciclo

PC antes de leer → IR formado → PC después del incremento → instrucción interpretada → cambios en registros/RAM/PC.



Paso 1 y 2: leer memoria y formar el IR

El PC funciona como un marcador: apunta al primer byte de la instrucción

00	01	02	03	04	05
20	C0	30	04	00	00



PC = 00

Se leen exactamente 2 bytes: mem[00] y mem[01].

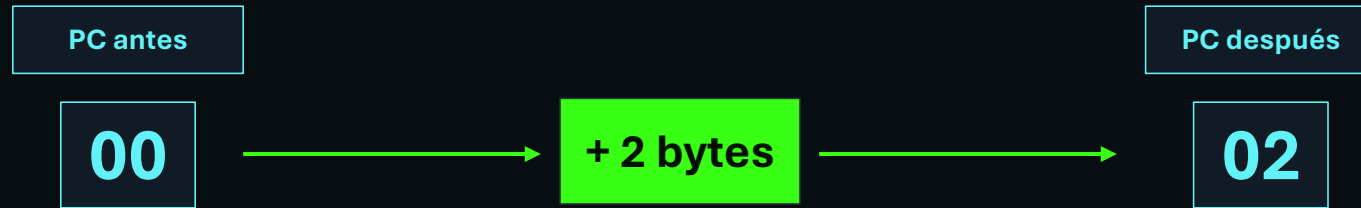
Luego se copian juntos en el Instruction Register.

IR = 20C0

No se ejecuta nada todavía: únicamente se trae la instrucción desde memoria.

Paso 3: incrementar el PC

Después de llenar el IR, el PC se mueve a la siguiente instrucción



¿Por qué +2? Porque cada instrucción ocupa dos posiciones consecutivas de memoria.

Paso 4: decodificar la instrucción

Separar la instrucción en campos evita confundirse

IR = 3004

OP

R

X

Y

código

registro

dirección / dato

3

0

0

4

OP=3 → STORE
R=0 → usar r0
XY=04 → dirección 04

Lectura correcta: guardar el contenido de r0 en la celda de memoria 04.

Paso 5: ejecutar y actualizar estado

Ejecutar significa cambiar registros, memoria o el PC según la instrucción

Instrucción	Antes	Acción	Resultado
LOADX 20C0	r0 = ??	$r0 \leftarrow C0$	r0 = C0
STORE 3004	r0 = C0 mem[04]=00	$mem[04] \leftarrow r0$	mem[04]=C0
JMP B021	r0 = r0	$PC \leftarrow 21$	Siguiente fetch en 21
HALT C000	Programa activo	Detener	No más ciclos

Idea de los ejercicios en lenguaje máquina VOLE



Plantilla de una línea: PC → IR → PC+2 → decodificar → ejecutar → revisar cambios → repetir.

4. Ejemplo 1: LOADX, STORE y HALT

Memoria inicial: 20 C0 30 04 00 00

00	01	02	03	04	05
20	C0	30	04	00	00

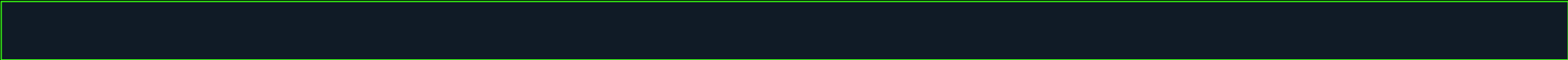
Ciclo 1: PC=00 → IR=20C0 → PC=02 → LOADX r0←C0.

Ciclo 2: PC=02 → IR=3004 → PC=04 → STORE mem[04]←r0 = C0.

Ciclo 3: PC=04 → IR=C000 → PC=06 → HALT detiene el programa.

PC	IR	PC luego	Ejecución
00	20C0	02	r0 = C0
02	3004	04	mem[04] = C0
04	C000	06	HALT

Estado final: r0=C0, mem[04]=C0, IR final=C000.



5. Ejemplo 2: LOAD desde memoria y STORE

Aquí LOAD no carga "02": carga el contenido ubicado en la dirección 02

00	01	02	03	04	05
14	02	34	17	C0	00

IR=1402: OP=1 LOAD, R=4, XY=02. Entonces $r4 \leftarrow \text{mem}[02]$. Como $\text{mem}[02]=34$, $r4=34$.

IR=3417: OP=3 STORE, R=4, XY=17. Entonces $\text{mem}[17] \leftarrow r4 = 34$.

IR=C000: HALT. El programa termina.

PC	IR	Ejecución	Cambio
00	1402	LOAD $r4 \leftarrow \text{mem}[02]$	$r4 = 34$
02	3417	STORE $\text{mem}[17] \leftarrow r4$	$\text{mem}[17] = 34$
04	C000	HALT	Fin

6. Ejemplo 3: antes del salto

El programa empieza en PC=20 y luego usa JMP para cambiar el flujo

20	21	22	23	24	25	26	27	28
12	20	32	30	B0	21	24	C0	00

PC	IR	PC luego	Ejecución
20	1220	22	LOAD r2 ← mem[20] = 12
22	3230	24	STORE mem[30] ← r2 = 12
24	B021	26 → 21	JMP: como r0=r0, PC ← 21

7. Ejemplo 3: después del JMP

El salto hace que el programa “reinterprete” bytes de memoria

20	21	22	23	24	25	26	27	28
12	20	32	30	B0	21	24	C0	00

PC	IR	PC luego	Ejecución
21	2032	23	LOADX r0 ← 32
23	30B0	25	STORE mem[B0] ← r0 = 32
25	2124	27	LOADX r1 ← 24
27	C000	29	HALT

Estado final: r0=32, r1=24, r2=12, mem[30]=12 y mem[B0]=32.

8. Ejemplo 4: operación OR a nivel de bits

El OR combina bits de dos registros y guarda el resultado

9C	9D	9E	9F	A0	A1	A2	A3	A4	A5	A6	A7
15	A7	26	80	75	56	35	A7	C0	00	71	54

IR=15A7 → LOAD r5 ← mem[A7] = 54

IR=2680 → LOADX r6 ← 80

IR=7556 → OR r5 ← r5 OR r6

IR=35A7 → STORE mem[A7] ← r5 = D4

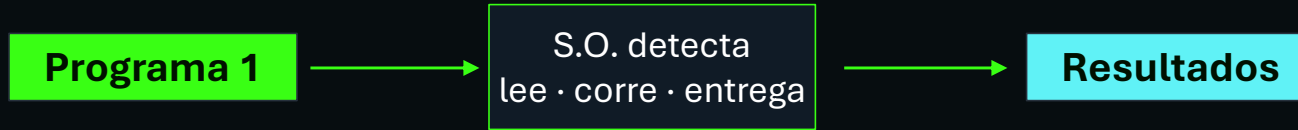
IR=C000 → HALT

0x54 = 0101 0100
0x80 = 1000 0000
OR = 1101 0100 = D4

Estado final: r5=D4, r6=80 y mem[A7]=D4.

10. Sistemas operativos primitivos y JCL

De ejecutar trabajos manualmente a controlar trabajos por tarjetas



Problema

¿Dónde termina un programa? ¿Qué compilador se usa? ¿Dónde empiezan los datos?

JCL

Job Control Language fue una interfaz primitiva para indicar al S.O. cómo ejecutar cada trabajo.

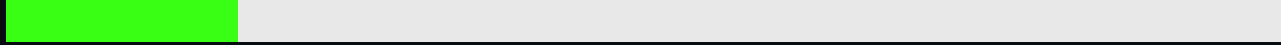
Antecesor de shell languages e interfaces más amigables.



11. JCL y procesamiento batch

La meta era aumentar el throughput: hacer más trabajos en menos tiempo

Máquina dedicada:



Operador batch:



Batch S.O.:



Con JCL, el usuario no conversaba con el sistema como hoy: debía preparar instrucciones de control para que el S.O. supiera qué hacer.

Idea central: automatizar el paso de un trabajo a otro redujo tiempo muerto entre programas.

12. Entrada/Salida primitiva

Antes, tratar con cada dispositivo era responsabilidad del programador



Como cada dispositivo podía tener su propio “lenguaje”, el programador debía conocer detalles de hardware y escribir rutinas específicas.

Con el tiempo, esas rutinas se compartieron y luego se integraron como bibliotecas de entrada/salida.

13. E/S + sistema operativo y servicios

La solución fue centralizar funciones comunes dentro del sistema operativo



Ventajas

La E/S se volvió más fácil y disponible para los programas.
El programador pide el servicio y recibe el resultado.

Costo

El S.O. crece en tamaño y complejidad. Centralizar introduce overhead porque las operaciones pasan por el sistema.

Servicios mencionados: device drivers y system calls como “instrucciones virtuales”.

14. Definiciones importantes

Términos que sostienen la idea de virtualización y lenguajes

Real

Se ve y existe

Virtual

Se ve, pero no existe físicamente

Transparente

No se ve, pero existe

Máquina

Dispositivo real, virtual o transparente que realiza una tarea computacional

Programa

Texto de símbolos que describe y prescribe el comportamiento de una máquina

Lenguaje

Notación para escribir programas y describir cálculos

Idea clave: cada máquina entiende directamente un lenguaje; las capas superiores construyen lenguajes más convenientes.

15. Sistema operativo como máquina virtual

“Una máquina que vemos pero no existe”

Los compiladores traducen para el sistema operativo; usan system calls en lugar de manipular directamente todo el hardware.



A cambio de alejarnos del hardware, ganamos conveniencia, seguridad, servicios y portabilidad.

Abstracción: una capa presenta una máquina más fácil de usar y oculta detalles de la capa inferior.

16. Máquinas multinivel y lenguajes

Más niveles aumentan conveniencia; menos niveles aumentan eficiencia

Conveniencia



Eficiencia ↑

Cada capa ofrece un lenguaje o interfaz más cercana al usuario o al problema. La máquina real queda debajo.

Cierre

Para resolver VOLE: sea ordenado, no salte pasos y actualice el estado después de cada instrucción.

PC → IR → PC+2 → decodificar → ejecutar → revisar cambios → repetir hasta HALT